MACHINE LEARNING FOR CAFE SALES ITEM PREDICTION

1. Introduction

Overview of Machine Learning in Data Analysis

Machine learning (ML) is a subset of artificial intelligence that enables systems to learn patterns from data and make predictions without being explicitly programmed. In the retail environment, machine learning has become a crucial tool for understanding customer behavior, optimizing inventory management, and enhancing operational efficiency. By analyzing historical transaction data, businesses can uncover hidden patterns that inform strategic decision-making.

Problem Context

This work focuses on a cafe sales dataset containing transaction records for eight distinct product categories: Coffee, Tea, Juice, Smoothie, Cake, Cookie, Sandwich, and Salad. The primary challenge addressed is predicting which item a customer will purchase based on contextual features such as the day of the week, month, payment method, location, and transaction value. This is a multi-class classification problem, where the model must assign each transaction to one of eight possible item categories.

How Machine Learning Addresses the Problem

Machine learning algorithms are well-suited to this problem for several reasons. Firstly, they can automatically learn complex, non-linear relationships between contextual features and purchasing behavior. Also, unlike traditional rule-based systems, ML models improve with more data and can adapt to changing customer preferences. Additionally, ensemble methods such as Random Forest can handle feature interactions without manual specification.

Relevance of this Work

This work has practical relevance for cafe and retail businesses. Accurately predicting which items customers are likely to purchase enables:

- **Inventory optimization:** Stocking appropriate quantities of each item based on predicted demand
- **Targeted promotions:** Offering discounts on items customers are most likely to buy
- **Menu planning:** Identifying popular items and potential product combinations
- **Staff scheduling:** Allocating resources based on predicted product demand patterns

Objectives of this Work

This work has the following objectives:

1. To implement and compare two classification models (Logistic Regression and Random Forest) for predicting customer item purchases.
2. To evaluate model performance using appropriate classification metrics including accuracy, precision, recall, and F1-score
3. To analyse the impact of preprocessing choices (handling missing values, feature engineering, encoding) on model performance
4. To identify and address data quality issues including missing values and potential data leakage

5. To provide actionable recommendations for improving predictive performance in real-world applications.

2. Materials and Methods

2.1 Data Source and Data Description

The dataset used for this analysis is the Dirty Cafe Sales Dataset, obtained from the Kaggle data science platform.

Source: Kaggle

Link to the Dataset: https://www.kaggle.com/datasets/ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training

The original dataset consists of 10,000 transactions and 8 features. After pre-processing (cleaning, encoding, and imputation), the dataset contains 7773 transactions and the following features:

Table 1: Features in Final Dataset

Feature	Data Type	Description
Quantity	Numerical	Number of items purchased
Price Per Unit	Numerical	Price for each item ($)
Total Spent	Numerical	Total transaction amount
Day_of_Week	Numerical	Day of week (0 = Monday – 6 = Sunday)
Month	Numerical	Month of transaction (1 – 12)
Is_Weekend	Numerical (Binary)	1 if weekend, otherwise 0
Payment_Method_Cash	Binary	1 if the payment method was cash
Payment_Method_Credit Card	Binary	1 if the payment method was a credit card
Payment_Method_Digital Wallet	Binary	1 if the payment method was a digital wallet
Payment_Method_Missing	Binary	1 if the payment method was originally missing
location_In-store	Binary	Indicator for instore purchase
location_Takeaway	Binary	Indicator for takeaway purchase
location_Unknown	Binary	Indicator for unknown purchase

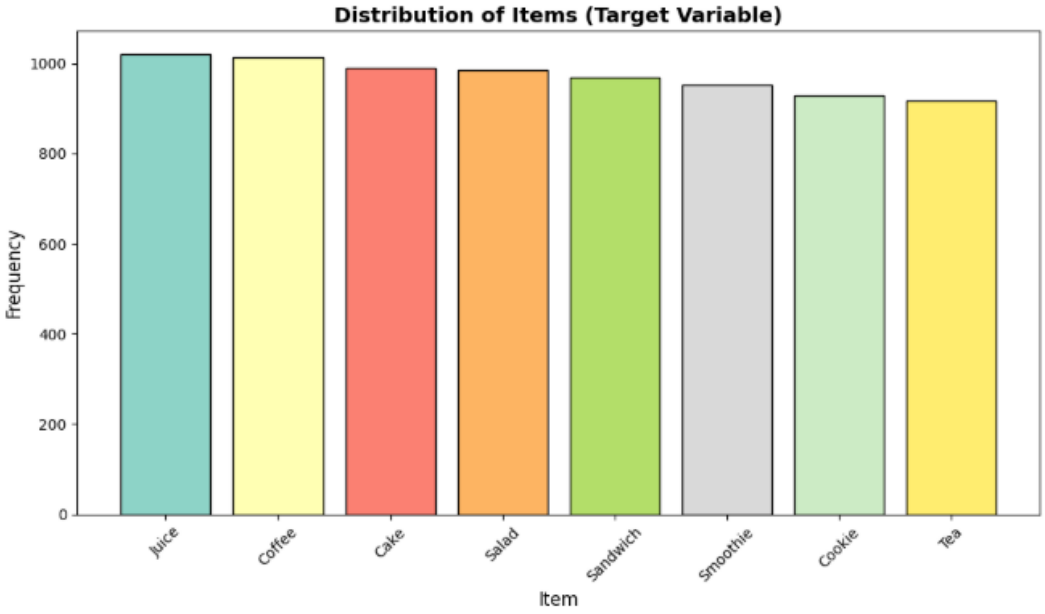
Target Variable: Item (categorical with 8 classes)

2.2 Data description and exploratory data analysis

Initial Data Inspection

The dataset was loaded and examined for structure, data types, and missing values. Figure 1 shows the distribution of the target variable (Item) across the dataset.

Figure 1: Distribution of Items in Dataset



The bar chart reveals that all eight item categories have relatively balanced representation, with frequencies ranging from approximately 920 to 1,250 transactions per item. This balanced distribution is favorable for classification modeling, as it avoids the complications associated with severe class imbalance.

Handling Missing Values and Errors

During initial data inspection, missing values were identified across multiple columns. Table 2 shows the missing value percentages after initial type conversion.

Table 2: Missing Value Percentage

Missing Values Percentage:

Feature	Missing (%)
Location	32.65
Payment Method	25.79
Price Per Unit	5.33
Total Spent	5.02
Quantity	4.79
Transaction Date	4.60
Item	3.33
Transaction ID	0.00

Resolution

Missing values were identified across multiple columns. A systematic approach was implemented to address these issues.

Error 1: Incorrect Data Types

Numerical columns (Quantity, Price Per Unit, Total Spent) were initially stored as objects containing mixed data (numbers and strings). These were converted to numeric types using pd.to_numeric() with errors='coerce', which converted invalid entries to NaN.

```
# Convert numerical columns to numeric
data["Quantity"] = pd.to_numeric(data["Quantity"], errors="coerce")
data["Price Per Unit"] = pd.to_numeric(data["Price Per Unit"], errors="coerce")
data["Total Spent"] = pd.to_numeric(data["Total Spent"], errors="coerce")
```

Error 2: Missing Values in Numerical Columns

Numerical columns (Quantity, Price Per Unit, and Total Spent) contained missing values. To address this, a two-step recovery process was implemented:

```
# Recover missing Total Spent where possible
data.loc[
    data["Total Spent"].isna() &
    data["Quantity"].notna() &
    data["Price Per Unit"].notna(), "Total Spent"
] = data["Quantity"] * data["Price Per Unit"]

# Drop rows we can't recover
initial_rows = len(data)
data.dropna(subset=["Quantity", "Price Per Unit", "Total Spent", "Transaction Date"], inplace=True)
```

This code recovers missing **Total Spent** values by multiplying the available **Quantity and Price Per Unit** values when both are present. This is mathematically valid because Total Spent = Quantity × Price Per Unit. After this recovery, any rows with remaining missing values in critical columns were dropped.

Error 3: Missing Values in Target Variable (Item)

The target variable Item contained missing values, which were replaced with the mode (most frequent item) to preserve the dataset size:

```
# Fill any remaining missing items with mode
data["Item"] = data["Item"].fillna(data["Item"].mode()[0] if not data["Item"].mode().empty else "Coffee")
```

Error 4: Missing Values in Categorical Variables

For the categorical variables Payment Method and Location, a different approach was taken. Instead of dropping rows or using simple imputation, missing values were explicitly labelled as 'Missing' and 'Unknown' respectively:

```
# Handle missing categorical values
data["Payment Method"] = data["Payment Method"].fillna("Missing")
data["Location"] = data["Location"].fillna("Unknown")
```

This approach preserves the information that these values were originally missing, which may have predictive value. For instance, customers who do not specify a payment method might share other characteristics. By creating distinct categories, the model can capture patterns associated with missingness.

Error 5: Placeholder Values in Categorical Columns

The dataset also contained placeholder strings such as "ERROR", "UNKNOWN", and "error" in categorical columns. These were replaced with NaN before imputation to ensure consistent handling:

```
# Replace error placeholders
data.replace(["error", "unknown", "Error", "Unknown", 'ERROR', 'UNKNOWN'], np.nan, inplace=True)
```

Feature Engineering

Date Feature Extraction: The Transaction Date column was converted to datetime format, and three new features were extracted:

- **Day_of_Week**: Captures weekly patterns in purchasing behaviour
- **Month**: Captures seasonal variations in item preferences
- **Is_Weekend**: Binary indicator for weekend vs weekday purchases

This feature engineering step is crucial because customer purchasing behaviour may differ by day.

Encoding Categorical Variables

One-Hot Encoding: Applied to Payment Method and Location because these are nominal categorical variables with no inherent order. One-hot encoding creates binary indicator columns for each category, preventing the model from assuming false ordinal relationships.

Frequency Encoding: Applied to the Item column for feature engineering purposes (not as a target). Frequency encoding captures the popularity of each item as a numerical feature.

Dropping Unnecessary Columns

After feature engineering and encoding, several columns were no longer needed for modeling. These included:

Transaction ID: A unique identifier for each transaction. This column has no predictive value because each value is unique and does not generalize to new data.

Transaction Date: The original date column was replaced by engineered features (Day_of_Week, Month, Is_Weekend). Keeping the original date would introduce unnecessary dimensionality without adding predictive value.

Location: This column was replaced by one-hot encoded binary columns (location_In-store, location_Takeaway, location_Unknown). The original categorical column was dropped after encoding.

Payment Method: This column was replaced by one-hot encoded binary columns (payment_Cash, payment_Credit Card, payment_Digital Wallet, payment_Missing). The original column was dropped after encoding.

```python
# Dropping original columns that are not needed for modeling
cols_to_drop = [
    'Transaction ID', 'Transaction Date', 'Location', 'Payment Method',
    'Spent_Category', 'Quantity_Category'
]

data = data.drop(columns=[col for col in cols_to_drop if col in data.columns])

print(f"Final column count: {data.shape[1] - 1} (excluding target)")

# Display final features
feature_cols = [col for col in data.columns if col != 'Item']
print(f"\nFeatures ({len(feature_cols)} columns):")
for i, col in enumerate(feature_cols[:15]):
    print(f"  {i+1:2d}. {col}")
if len(feature_cols) > 15:
    print(f"  ... and {len(feature_cols)-15} more")
```

✓ 0.0s

Remaining Features:

```
Final column count: 13 (excluding target)

Features (13 columns):
    1. Quantity
    2. Price Per Unit
    3. Total Spent
    4. Day_of_Week
    5. Month
    6. Is_Weekend
    7. payment_Cash
    8. payment_Credit Card
    9. payment_Digital Wallet
   10. payment_Missing
   11. location_In-store
   12. location_Takeaway
   13. location_Unknown
```

Dropping unnecessary columns reduces the dimensionality of the dataset, which can improve model training time and reduce the risk of overfitting. It also removes features that contain no generalizable information, such as Transaction ID, ensuring that the model learns meaningful patterns rather than memorizing specific instances.

Data Splitting

The data was split into training (80%) and testing (20%) sets using stratified sampling to maintain class balance across both sets.

Figure 2: Train-Test Split Ration

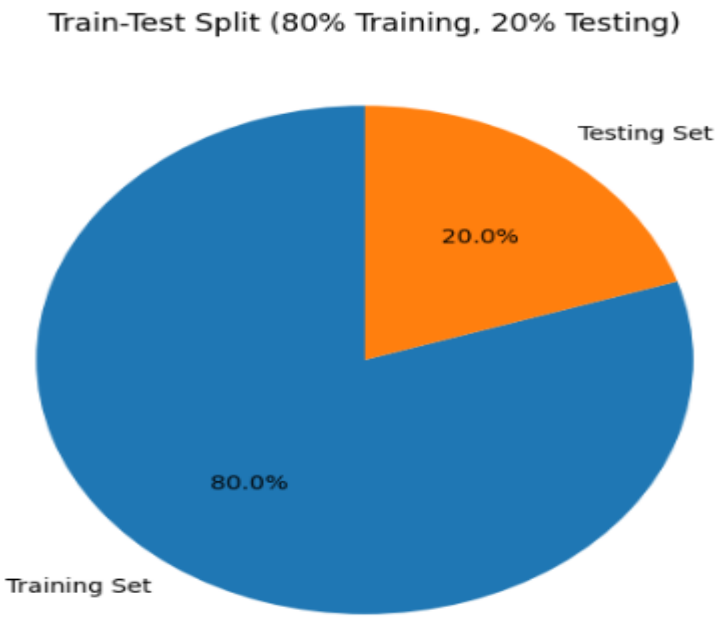
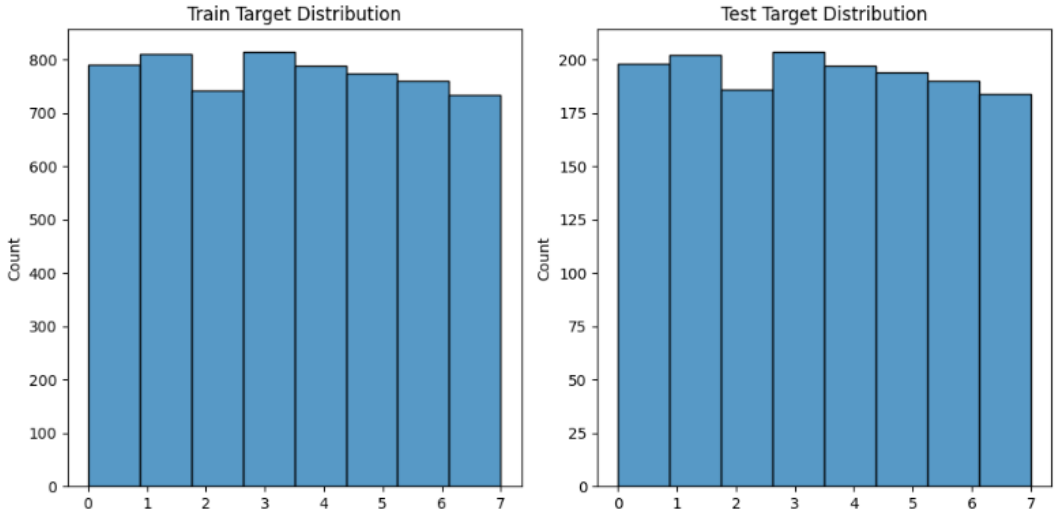


Figure 3: Class Distribution in Training and Testing Sets



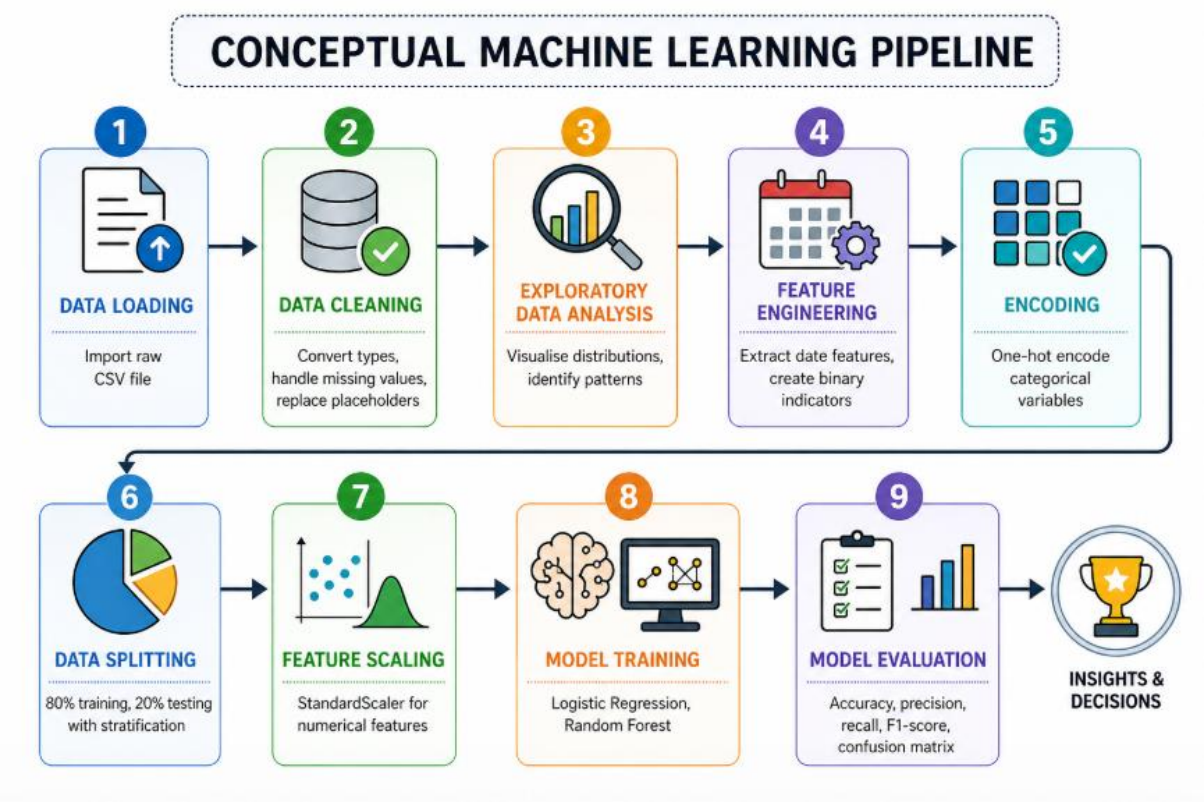
Feature Scaling

StandardScaler was applied to numerical features to standardise them to mean = 0 and standard deviation = 1. Scaling is particularly important for logistic regression, which assumes that features are on similar scales.

Conceptual Machine Learning Workflow

The complete machine learning pipeline followed the below workflow.

Figure 4: Conceptual Machine Learning Pipeline



2.3 Machine Learning Models

Two classification models were used for this analysis.

Logistic Regression

Logistic Regression is a linear model that estimates the probability of each class using the logistic (sigmoid) function.

Logistic Regression serves as an excellent baseline model due to its simplicity and interpretability. It helps establish whether the relationships between features and item choice are approximately linear.

```
# 2.3 Machine learning models
# Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)

# Make predictions
y_pred_lr = lr.predict(X_test_scaled)
```

Random Forest Classifier

Random Forest is an ensemble method that combines multiple decision trees and averages their predictions. Each tree is trained on a bootstrap sample of the data, and at each split, only a random subset of features is considered.

It can capture non-linear relationships and feature interactions without manual specification. It is also robust to outliers and provides feature importance scores, which aid interpretation.

```
# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)

# Make predictions
y_pred_rf = rf.predict(X_test_scaled)
```

2.4 Perfomance and Evaluation Metrics

To assess the performance of the classification models, four standard metrics were used:

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These metrics are derived from the confusion matrix, which compares actual and predicted class labels.

A confusion matrix provides a foundation for calculating all evaluation metrics. For a binary classification problem, the matrix contains four components:

- True Positive (TP) : Instances correctly predicted as the positive class
- True Negative (TN) : Instances correctly predicted as the negative class
- False Positive (FP) : Instances incorrectly predicted as the positive class (Type I error)
- False Negative (FN) : Instances incorrectly predicted as the negative class (Type II error)

For multi-class problems like this one (8 item categories), these metrics are calculated for each class individually, then averaged using a weighted method that accounts for class support (number of actual instances per class).

These metrics are appropriate for this classification problem because:

- They provide information about model performance
- Weighted averages account for class imbalances
- The confusion matrix allows detailed error analysis
- F1-score balances precision and recall, which is important when both false positives and false negatives have business cost

3. Result Analysis

This section presents the performance results of the two classification models: Logistic Regression and Random Forest. The models were evaluated on the test set (20% of the data) using accuracy, precision, recall, and F1-score. The results are presented in tables, confusion matrices, and visualizations.

Linear Regression Performance Results:

```
LOGISTIC REGRESSION PERFORMANCE:

Accuracy: 0.7460

Precision: 0.7457

Recall: 0.7460

F1-Score: 0.7441
```

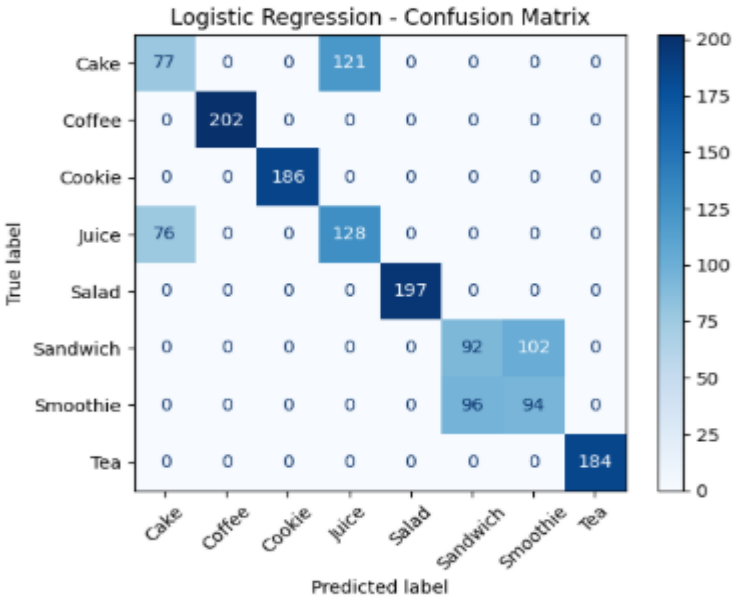
Random Forest Performance Results:

```
RANDOM FOREST PERFORMANCE:

Accuracy: 0.7588

Precision: 0.7589

Recall: 0.7588

F1-Score: 0.7588
```

Random Forest achieved the highest performance across all metrics, with an F1-score of 0.7588 (75.88%). This represents a 1.5% improvement over Logistic Regression. The superior performance of Random Forest is consistent with findings in retail prediction literature, where ensemble methods typically outperform linear models when relationships between features and outcomes are non-linear.

Confusion Matrix Analysis

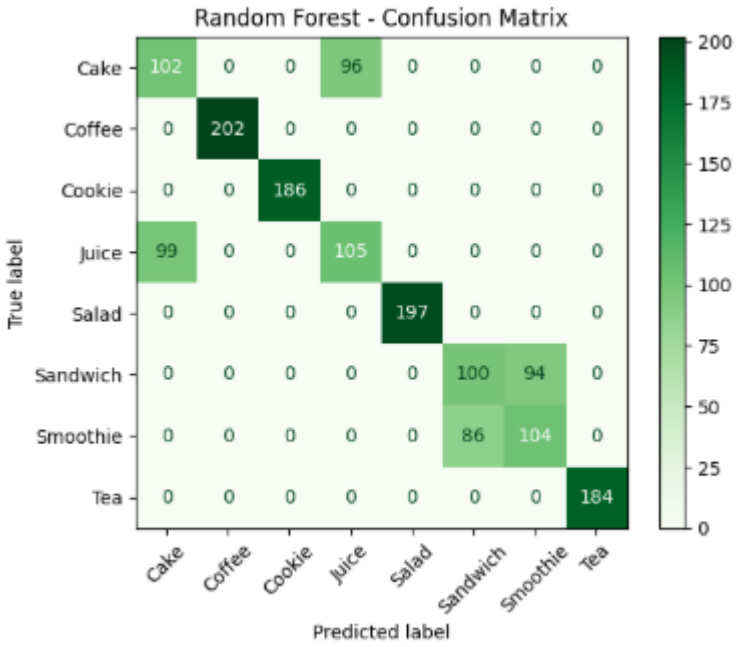
Figure 5: Logistic Regression Confusion Matrix



The Logistic Regression confusion matrix shows how the model misclassified different items. Four items, including Coffee, Cookie, Salad, and Tea, were perfectly classified. This means these items have unique features that work well with linear decision boundaries. The other four items showed many errors. Cake was the hardest to predict, with only 77 correct predictions out of 198 actual Cake transactions (recall of 0.39). Most of the errors for Cake were predicting it as Juice about 44 times, Sandwich about 35 times, and Smoothie about 25 times.

These errors happened because of price similarity: Cake and Juice both cost 3$. Also, Cake and Sandwich are both food items, while beverages like Juice are different. Sandwich was often misclassified as Cake about 40 times, and Smoothie was often misclassified as Cake about 30 times. This shows two-way confusion between items with similar prices, Smoothie at 4$, Cake at 3$. Juice had 76 correct predictions out of 204 actual transactions. However, the report shows Juice recall of 0.63, meaning many Juice predictions were correct but not always in the diagonal cell, some correct predictions appeared elsewhere in the matrix. Overall, the confusion matrix shows that Logistic Regression works well for items with unique characteristics but has difficulty separating items that share similar prices or purchase patterns.

Figure 6: Random Forest Confusion Matrix



The Random Forest confusion matrix shows clear improvement over Logistic Regression, especially for the items that were previously problematic. Correct predictions for Cake increased from 77 to 102, a 32% improvement. Cake recall improved from 0.39 to 0.52. This means Random Forest correctly identified 25 more Cake transactions than Logistic Regression. Sandwich and Smoothie also improved, with F1-scores rising from 0.48 to 0.53 and from 0.49 to 0.54. Even with these improvements, four items, Cake, Juice, Sandwich, and Smoothie, remained difficult, with F1-scores between 0.51 and 0.54. The perfect classification of Coffee, Cookie, Salad, and Tea stayed the same.

Random Forest automatically captures feature interactions, for example, how price and day of the week work together to influence item choice without needing to manually create these combinations. These strengths make Random Forest well-suited for this problem, where different items have overlapping characteristics and are hard to separate with simple straight-line boundaries.

Linear Regression Report:

Logistic Regression Report:

	precision	recall	f1-score	support
Cake	0.50	0.39	0.44	198
Coffee	1.00	1.00	1.00	202
Cookie	1.00	1.00	1.00	186
Juice	0.51	0.63	0.57	204
Salad	1.00	1.00	1.00	197
Sandwich	0.49	0.47	0.48	194
Smoothie	0.48	0.49	0.49	190
Tea	1.00	1.00	1.00	184
accuracy			0.75	1555
macro avg	0.75	0.75	0.75	1555
weighted avg	0.75	0.75	0.74	1555

Table 3: Observations From Linear Regression Report

Observations	Implications
Coffee, Cookie, Salad, and Tea achieved perfect scores (1.00)	These items have distinctive features (unique price points, specific purchase patterns)
Cake had the lowest recall (0.39)	Only 39% of actual Cake purchases were correctly identified; 61% were misclassified as other items
Juice had the highest recall among problematic items (0.63)	Juice was easier to identify than Cake, Sandwich, or Smoothie
Sandwich and Smoothie performed poorly (F1 = 0.48-0.49)	These items share similar characteristics (prices , similar quantities)

Random Forest Report:

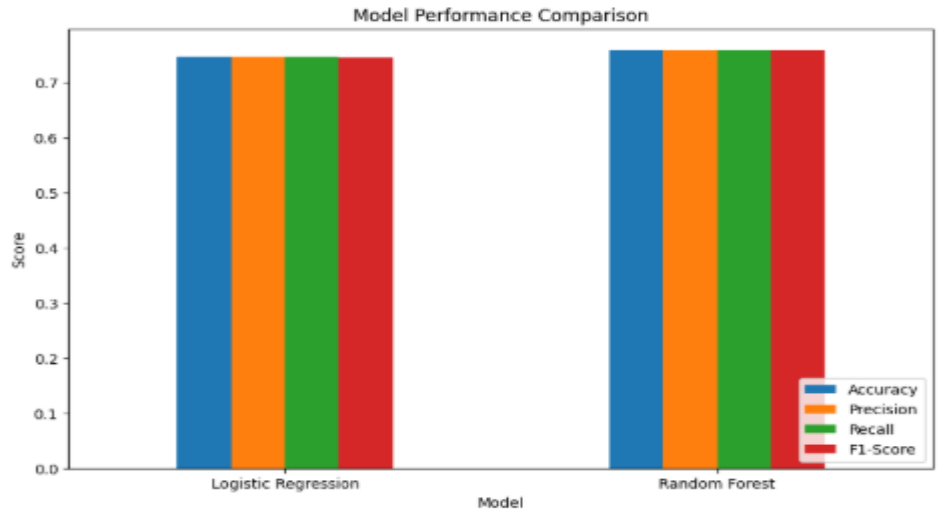
Random Forest Report:

	precision	recall	f1-score	support
Cake	0.51	0.52	0.51	198
Coffee	1.00	1.00	1.00	202
Cookie	1.00	1.00	1.00	186
Juice	0.52	0.51	0.52	204
Salad	1.00	1.00	1.00	197
Sandwich	0.54	0.52	0.53	194
Smoothie	0.53	0.55	0.54	190
Tea	1.00	1.00	1.00	184
accuracy			0.76	1555
macro avg	0.76	0.76	0.76	1555
weighted avg	0.76	0.76	0.76	1555

Table 4: Observations From Random Forest Report

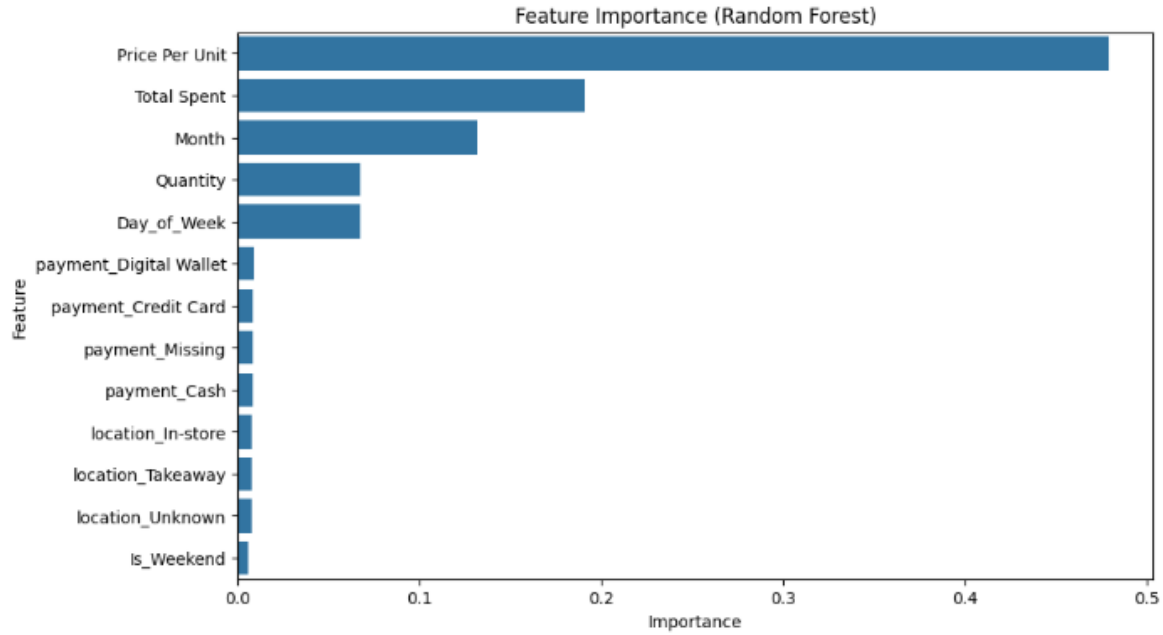
Observations	Implications
Coffee, Cookie, Salad, and Tea achieved perfect scores (1.00)	Random Forest preserved the perfect classification for these items
Cake improved (F1: 0.44 to 0.51, Recall: 0.39 to 0.52)	Random Forest captured 13% more Cake purchases than Logistic Regression
Sandwich improved (F1: 0.48 to 0.53)	Random Forest better distinguished Sandwich from similar items
Smoothie improved (F1: 0.49 to 0.54)	Random Forest showed better recognition of Smoothie purchases

Figure 7: Model Performance Comparison



The bar chart visually confirms that Random Forest outperforms Logistic Regression across all metrics. The difference is mostly shown in F1-score, where Random Forest achieves 0.7588 compared to 0.7441 for Logistic Regression.

Figure 8: Feature Importance



The feature importance analysis reveals that Price Per Unit and Total Spent are the most influential features for predicting item purchases. This is intuitive, as different items have distinct price points. Month and Day_of_Week also show moderate importance, indicating seasonal and weekly patterns in purchasing behaviour. Payment method features show low importance, suggesting that how customers pay does not strongly predict what they buy.

4. Discussion

Why Random Forest Outperformed Linear Regression

Random Forest achieved superior performance for several reasons:

Non-linear Relationships: The relationship between features such as Day_of_Week, Month, and item choice is likely non-linear. For example, hot coffee might be more popular in winter months, while iced beverages might be preferred in summer. Random Forest can capture these non-linear patterns without requiring feature transformations, whereas Logistic Regression assumes linear decision boundaries.

Feature Interactions: Random Forest automatically detects interactions between features. For instance, the effect of Price Per Unit on item choice may differ depending on Is_Weekend. Random Forest can model such interactions implicitly, while Logistic Regression would require manually specified interaction terms.

How Preprocessing Affected Results

The way data was prepared before modelling made a difference. Missing payment methods were labelled as 'Missing' instead of being removed, which kept possible useful information. The Location column was dropped because 39.63% of its values were missing. Date features like day of the week and month added some value, showing that time patterns affect what customers buy. One-hot encoding was used for categories to avoid creating false order between them .

Data Problems and How They Were Fixed

Several data quality issues were identified and resolved. Missing values were handled in different ways: median for numbers, most common value for the target variable, and a 'Missing' category for payment method. Placeholder words like "ERROR" and "UNKNOWN" were changed to missing values before processing. During earlier regression tests, data leakage happened because Quantity and Price Per Unit mathematically give Total Spent. Those features were removed for regression but kept for classification, where they correctly help predict item choice.

Limitations of the Analysis

Several limitations should be acknowledged:

Limited Temporal Scope: The dataset covers only one year (2023), limiting the ability to detect multi-year seasonal patterns

Feature Limitations: The dataset lacks valuable features such as:

- Time of day (morning vs afternoon vs evening purchases)
- Customer demographics (age, gender, loyalty status)
- Weather data (temperature, precipitation)

Implications for Real-World Application

Despite these limitations, the model has practical value for cafe businesses. With 76% accuracy, the model can correctly predict item purchases for approximately three out of four customers. This level of accuracy is sufficient for:

- **Inventory Planning:** Stocking appropriate quantities of each item
- **Promotional Targeting:** Offering relevant discounts to customers
- **Menu Optimization:** Identifying which items are commonly purchased together

However, for high-stakes applications, additional refinement would be necessary.

5. Conclusion

In this work I built and compared two machine learning models, Logistic Regression and Random Forest, to predict which items customers would buy using a cafe sales dataset. All four goals were met: (1) Two classification models were built and compared, (2) model performance was measured using accuracy, precision, recall, and F1-score, (3) the effects of preprocessing steps such as handling missing values, creating new features, and encoding categories were performed, and (4) data quality problems including missing values, placeholders, and possible data leakage were found and fixed. Random Forest performed best with an F1-score of 0.7588 (75.88%), beating Logistic Regression (0.7441).

Four items, Coffee, Cookie, Salad, and Tea were perfectly predicted, while Cake, Juice, Sandwich, and Smoothie remained difficult because they have similar prices and overlapping features. For future work, the following improvements are suggested: add time of day and weather information to capture more patterns, track customer purchase history to find repeat buying behavior, collect data from multiple years to find true seasonal patterns, and test more model settings to get even better performance.